No More Big Bang: Make Your Releases Bulletproof

Marek Dominiak



Marek Dominiak

I run training in EventStorming, Architecture, DDD, and TDD

19 years of professional experience

Co-owner and instructor at trainitek

Hands-on Architect and Team Lead at MINTRA

https://www.linkedin.com/in/marekdominiak

Have you ever…

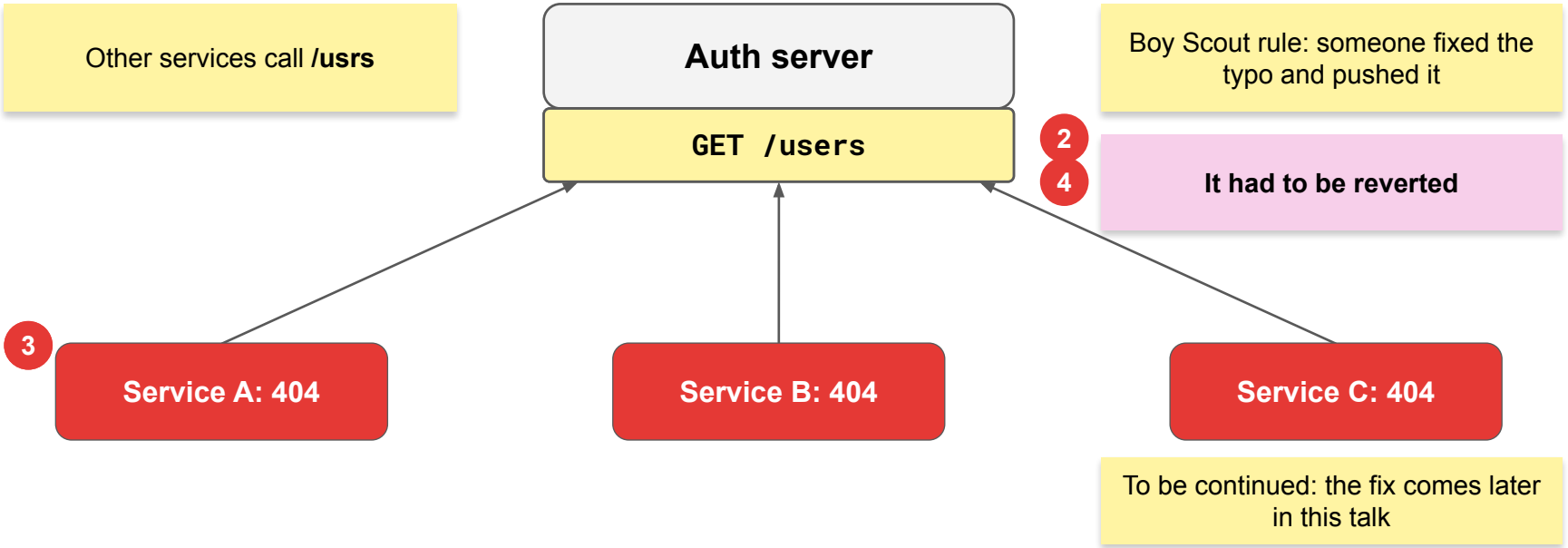
- Needed to stop a release and wait until other parts of the system got released first?
- Needed to roll back because your API wasn't compatible with the consumers?
- Experienced code freezes?
- Lived through a "big bang" release?



Good news: in most situations, you can avoid those issues.

Release #1: fixing a typo in an endpoint

Many years ago, a company started to work on microservices



Renaming an endpoint is a breaking change.

Agenda

- What is a bulletproof release?
- Do we need zero downtime?
- Five releases that went wrong, and how to fix them
- The tip of the iceberg



What is a bulletproof release?

Bulletproof releases

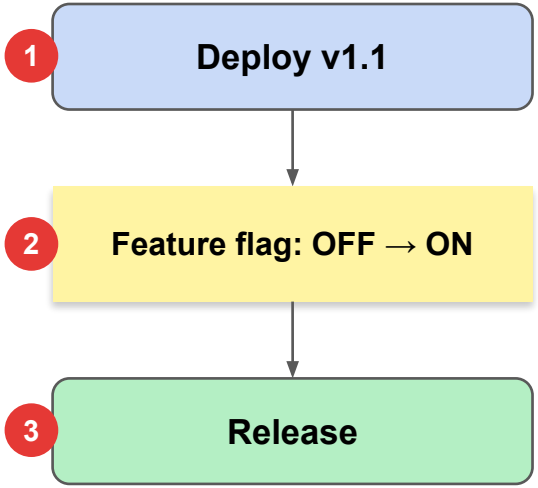
- We deploy whenever we want, no code freezes
- We don't wait for other teams to release first
- Users don't notice the deployment
- Old and new versions can run side by side
- If something goes wrong, going back is quick and cheap



Not "nothing ever breaks". When it breaks, it's cheap.

Deployment is not a release

- **Deployment:** new code runs in production
- **Release:** users get the new behavior
- Feature flags split the two
- Turning a flag off is usually faster than a rollback

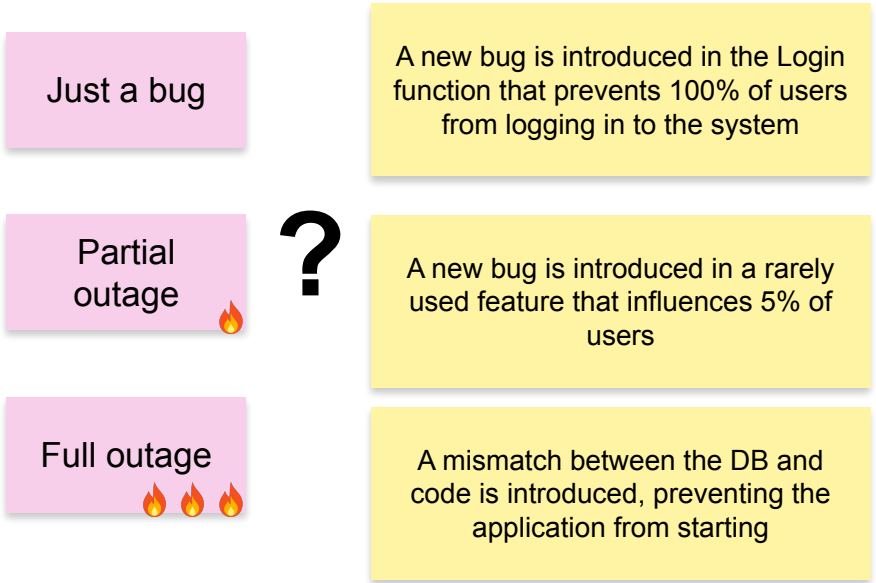


Deploy often. Release when you're ready.

How do we define downtime/outage?

We should define what is considered "downtime" (or an outage).

This should be agreed in SLA.



Agree on it before the incident, not during it.

Do we need zero downtime?

Zero downtime - do we need it?

We need it

Strict SLAs & Compliance: Enterprise or regulated environments (finance, healthcare) requiring guaranteed uptime

Customer-Facing, High-Availability Services: E.g.: e-commerce, streaming where any outage impacts revenue or user trust

Global or Real-Time Systems: Applications with users across time zones or continuous data streams (e.g. crypto trading) that can't pause

We may not need it

Non-critical features that can wait for off-peak deploys

Internal tools or **admin interfaces** that are used only during business hours

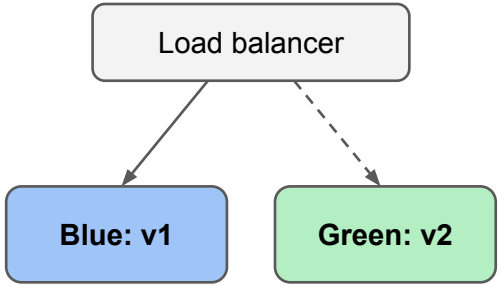
Low-traffic or **batch-oriented** applications where offline windows are acceptable

When **maintenance window** is available

Zero downtime has a price. Check if you need to pay it.

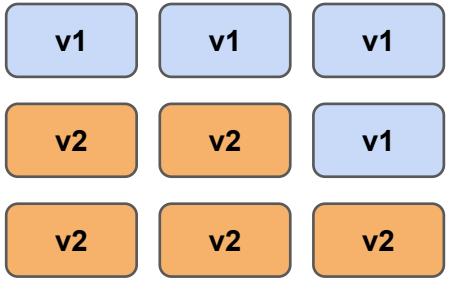
Zero downtime deployment strategies

Blue-green



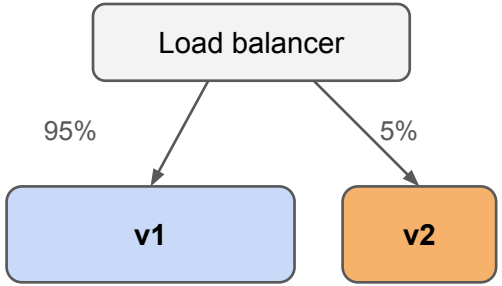
Two production environments. Test green, switch traffic. Blue is kept as a quick fallback.

Rolling updates



Replace instances in small batches. Some instances keep serving traffic at all times.

Canary

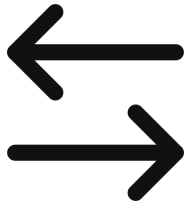


A small subset of users or servers first. Expand only if metrics stay healthy.

In all three, old and new versions run side by side.

Compatibility concepts

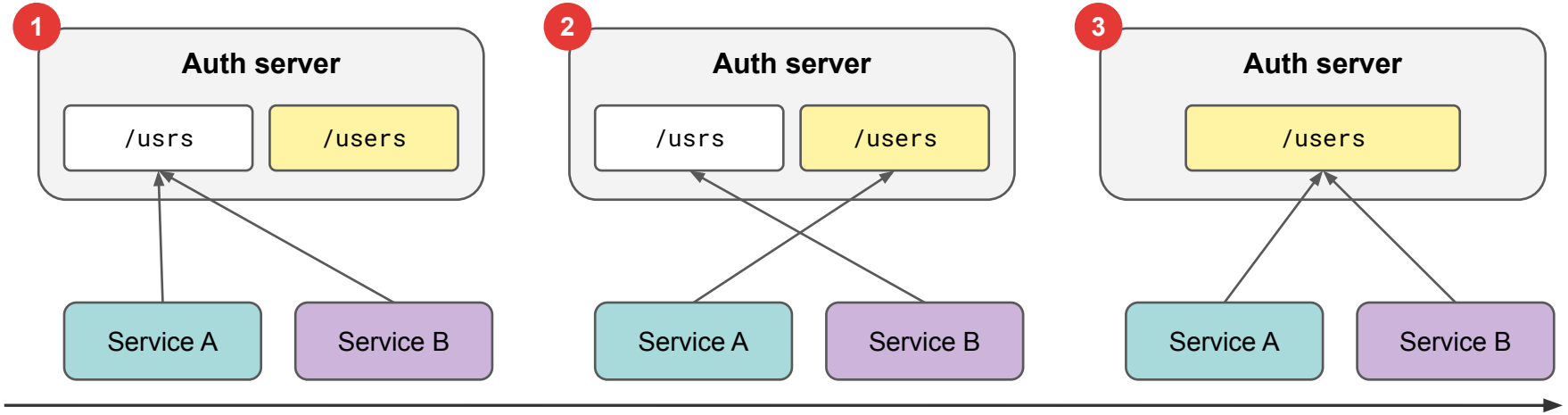
- **Backward compatibility:** new code reads old data. v2 reads the rows and messages v1 wrote.
- **Forward compatibility:** old code reads new data. v1, still running (or rolled back), reads what v2 writes.
- **Full compatibility:** both directions at the same time.



Old and new versions run side by side, so we need both directions.

Five releases that went wrong

Back to release #1: expand, migrate, contract



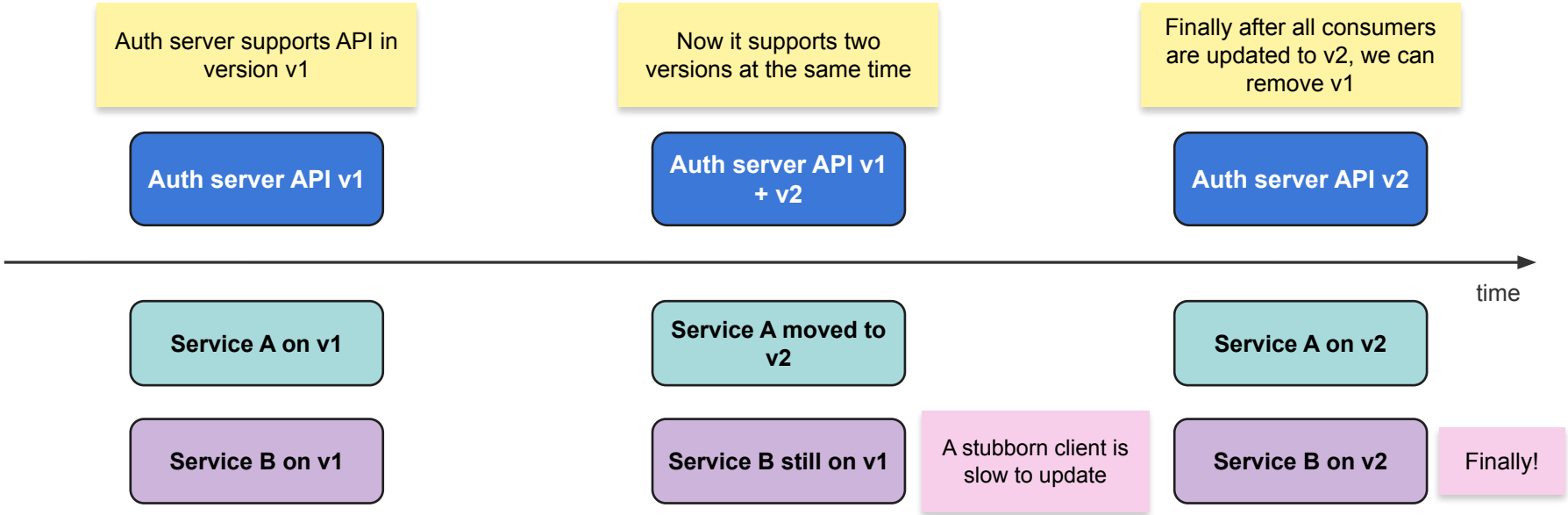
Expand: add /users next to /usrs. An alias is enough: same code, two routes.

Migrate: consumers move at their own pace. Watch the traffic on /usrs.

Contract: no traffic on /usrs? Remove it.

Add the new endpoint first. Remove the old one last.

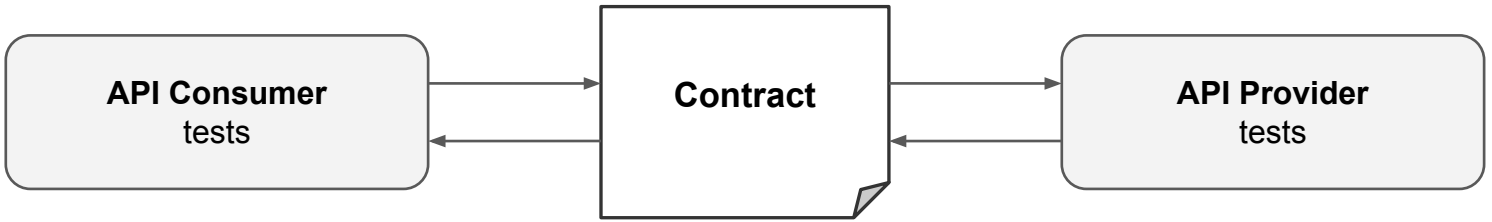
Multi versions support



Keep the number of supported versions low, e.g. 2.

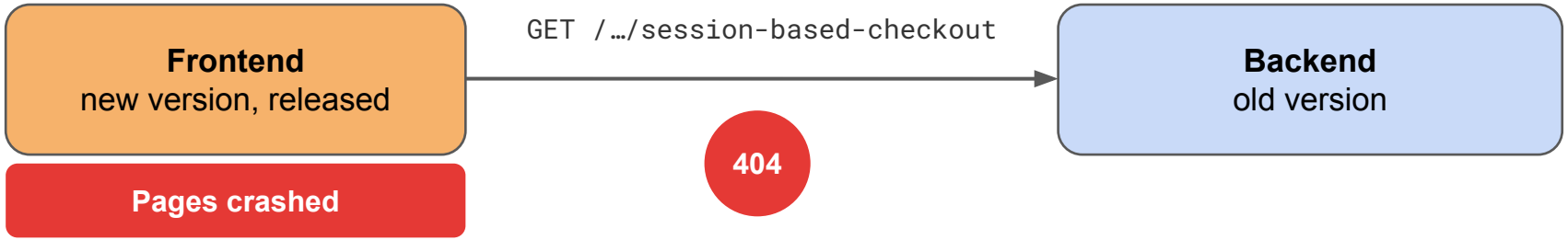
Contract Tests

- Check API-based communication between services.
- Must be provided for both sides (API provider and API consumer).
- Tools: OpenAPI, Pact (pact.io), Spring Cloud Contract
- Release #1 would have failed in CI, not in production



Tools cost too. Sometimes accepting the risk is cheaper.

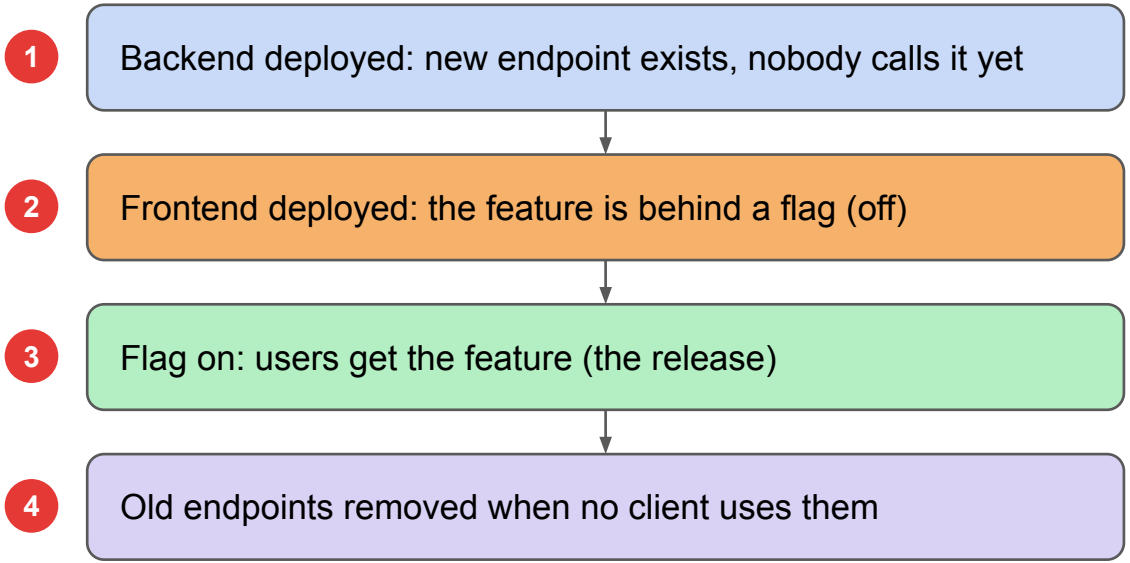
Release #2: the frontend was faster



1. Our frontend was released much quicker than our backend changes
2. The frontend assumed the new endpoints already existed
3. Some pages crashed, some functions couldn't work
4. Hotfixes: we reverted the related changes

The frontend depended on a backend that wasn't there yet.

The fix: backend first, frontend behind a flag



Tolerant frontend: a missing endpoint hides the feature, it doesn't crash the page

Contract tests catch a missing endpoint in CI

Flag off? The frontend can even ship before the backend.

Flag off: deploy in any order. Flag on: backend first.

Tolerant Reader

"… be as tolerant as possible when reading data from a service …" - Martin Fowler

- Read only the fields you need
- Ignore unknown fields
- Use defaults for missing optional fields
- Add fields, don't rename or remove them

```
// Provider v2 adds a new field
{
  "userId": 123,
  "email": "user@example.com",
  "plan": "premium"  // new
}

// Consumer v1 reads only what it needs
@JsonIgnoreProperties(ignoreUnknown = true)
record User(Long userId, String email) {}
// "plan" is ignored. Nothing breaks.
```

We should update our APIs in a Non-breaking way.

Ref: https://martinfowler.com/bliki/TolerantReader.html

Async APIs - Breaking changes

- Renaming, removing, or adding required fields.
- Changing data types (e.g., int → string).
- Messages already in the queue were produced by the old version.

Old clients won't be able to parse the payload

Old payload

```
{
  "userId": 123,
  "email": "user@example.com",
  "portalId": 25,
  "membership_tier": "premium"
}
```

New payload




```
{
  // Renamed field (userId → id)
  "id": 123,
  "email": "user@example.com",
  // portalId changed to string
  "portalId": "25"
  // Membership tier removed!
}
```

In async, old messages outlive the deployment.

Release #3: the database changed before the code

- A column removed too early:
  - New nodes didn't start, Hibernate schema validation failed
  - Running nodes threw runtime exceptions
- A data script ran pre-deploy instead of post-deploy:
  - Admins got a new authority the running code didn't know
  - Login failures, data deleted manually in production



The running code and the database got out of sync.

Example: renaming a column during a rolling update

```
CREATE TABLE orders (
  id      INT PRIMARY KEY,
  statuss VARCHAR(10) NOT NULL
);
```

The **statuss** column has a typo, we want to fix it

Adding orders fails because v1.0 isn't aware of a renamed column:
Unknown column 'statuss' in 'field list'

Rolling update

Orders Service - v1.0

Orders Service - v1.0

Adding orders works

Orders Service - v1.0

Orders Service - v1.1

Orders Service - v1.1

Orders Service - v1.1

Adding orders works again

time

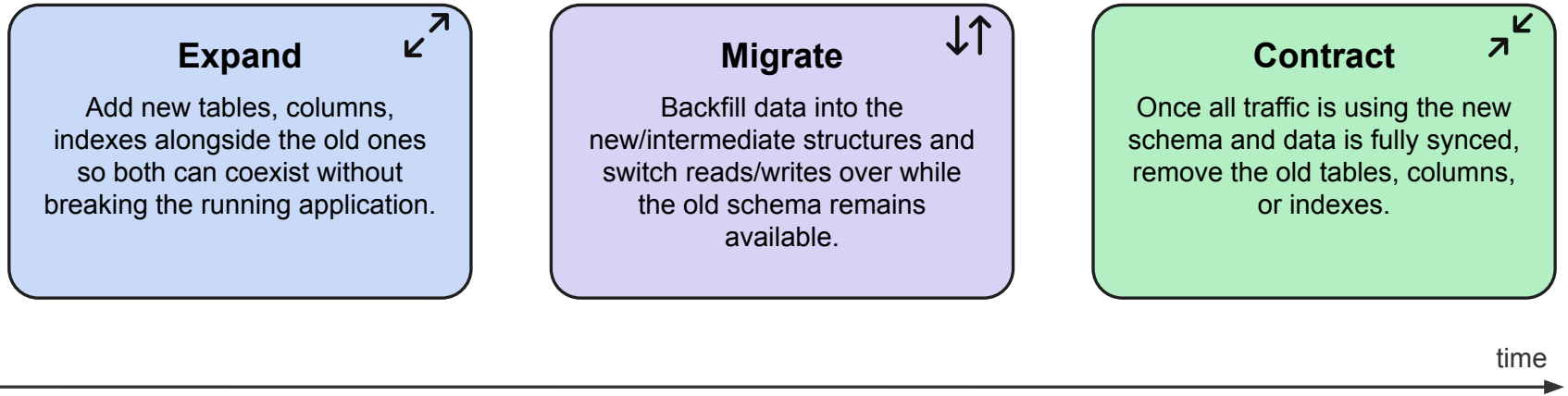
1. Rollout started
2. Db migration run
3. First node updated
4. Second node updated

```
ALTER TABLE orders
RENAME COLUMN statuss TO status;
```

Service is partially impaired for a period of time

The fix: expand, migrate, contract for data

- **Zero-downtime** remains our primary objective. To support this, we can use the **Expand, Migrate, and Contract** strategy (it's useful in many cases, not only for Zero-Downtime)



Expand, Migrate and Contract - Example

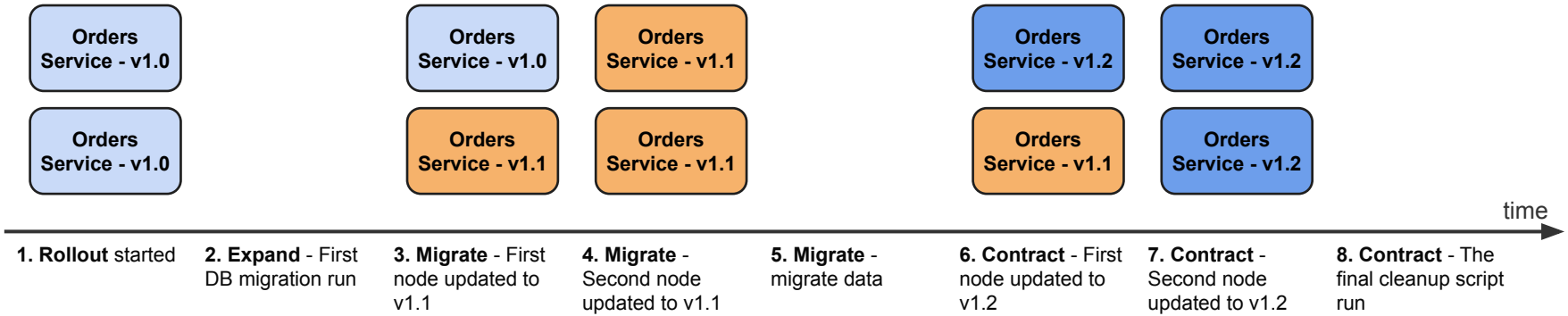
Adding orders works

v1.1 writes to **both** "status" and "statuss" columns

v1.1 reads from both "status" and "statuss" columns

v1.2 clean-up: writes and reads only the "status" column

At all times adding outstanding orders works



1. **Rollout** started
2. **Expand** - First DB migration run
3. **Migrate** - First node updated to v1.1
4. **Migrate** - Second node updated to v1.1
5. **Migrate** - migrate data
6. **Contract** - First node updated to v1.2
7. **Contract** - Second node updated to v1.2
8. **Contract** - The final cleanup script run

```
ALTER TABLE orders
ADD COLUMN status VARCHAR(10) NULL;
```

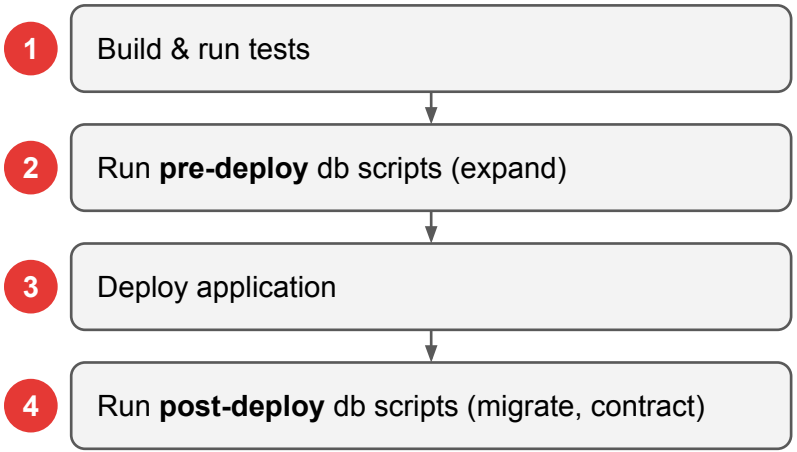
```
UPDATE orders SET status = statuss
  WHERE status IS NULL;
ALTER TABLE orders MODIFY COLUMN
  status VARCHAR(10) NOT NULL;
ALTER TABLE orders MODIFY COLUMN
  statuss VARCHAR(10) NULL;
```

```
ALTER TABLE orders
DROP COLUMN statuss;
```

Example: Migration with Flyway

Directory structure:

```
migrations/
├── pre-deploy/
│   └── V1.1.0__add_status_column.sql
└── post-deploy/
    ├── V1.1.1__backfill_status.sql
    ├── V1.1.2__enforce_status_not_null.sql
    ├── V1.1.3__relax_statuss_nullable.sql
    └── V1.2.0__drop_statuss_column.sql
```



Running a specific pre/post deployment migration

```
flyway -url="${DB_URL}" -locations=filesystem:migrations/${PHASE} \
  -ignoreMigrationPatterns="*:missing" migrate
```

PHASE: pre-deploy or post-deploy

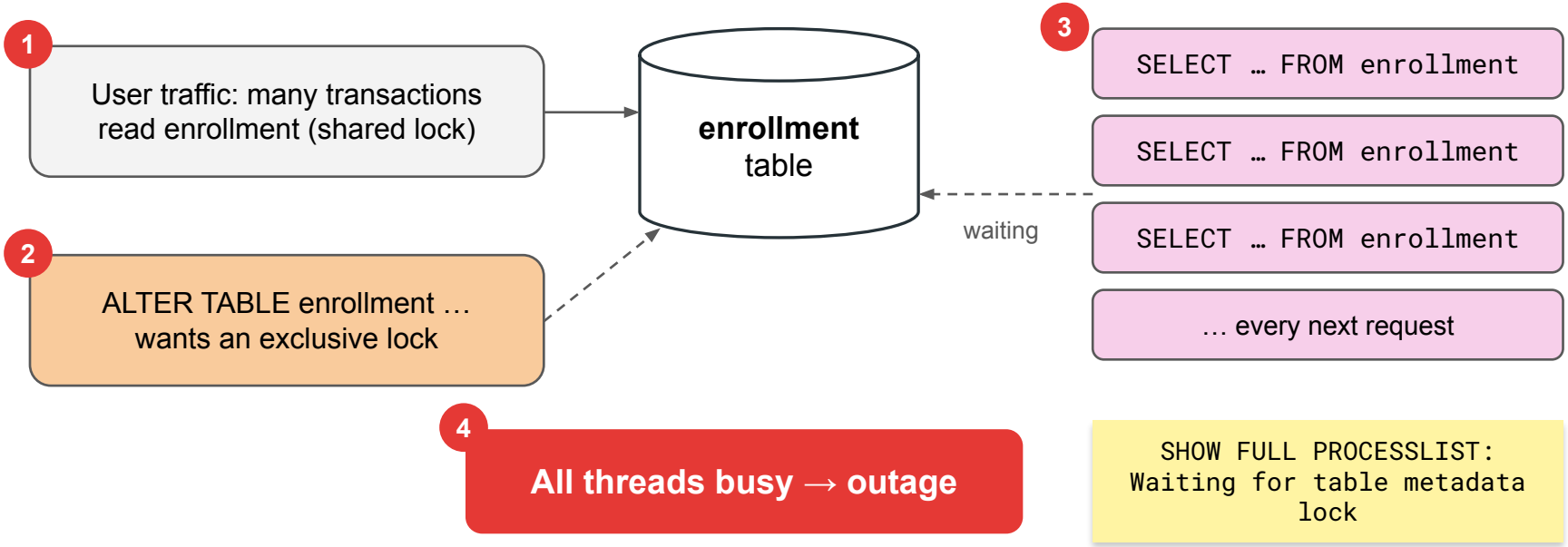
Pre-deploy: only changes the running code can live with.

Release #4: a database migration

- We wanted to ALTER the enrollment table during a deployment
- Normal traffic: every user running a course reads enrollments
- Many transactions held a metadata lock on that table
- The ALTER waited for the lock, with no timeout
- New queries queued behind it, within about a minute all threads were taken
- A stressful outage, and weeks of making up for the lost uptime



How a metadata lock takes the system down



A waiting ALTER blocks everyone behind it.

How we recovered

1. Killed the hanging transaction with the ALTER TABLE
2. The queued transactions were released
3. Added timeouts to our migrations
4. Ran the migration in our maintenance window



Kill the waiting ALTER first. It releases everyone behind it.

The fix: every migration needs a timeout

- MySQL default lock_wait_timeout: 31,536,000 s (1 year!)
- Set a short timeout in the migration, fail fast (we use 10 s in production)
- Retry later instead of blocking everyone
- Check for long-running transactions before you migrate

```
-- MySQL
SET SESSION lock_wait_timeout = 10;
ALTER TABLE orders
  ADD COLUMN status VARCHAR(10) NULL;

-- PostgreSQL
SET lock_timeout = '10s';
```

Fail fast and retry, instead of waiting forever.

The fix: a maintenance window for scary changes

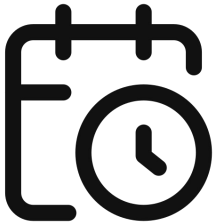
Maintenance Window: Scheduled downtime slot for upgrades, schema migrations, backups, and deep system checks.

Use it for **high-risk or large-scale** work that's hard to split into non-disruptive steps

Off-peak: nights or weekends with the lowest/no usage

Announce well in advance and give progress updates

Automate pre-checks and post-validation to keep the window short



Scary migrations/changes: plan a maintenance window.

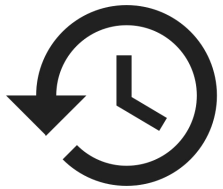
Release #5: rewriting our indexer

- We rewrote the indexer we wrote 14 years ago (V2)
- Test coverage close to 100% + performance tests
- Beta, then staging, then production
- The production traffic was much higher than anything we tested
- We rolled forward with fixes
- Eventually we needed to roll back



Why the rollback hurt

- I trusted V2 too much: no feature flag
- V2 reused and extended V1's tables, and I removed the old indexer
- Rolling back meant preparing V1 again, in a hurry
- A lovely 10 hours after work
- We stabilized the system, and finally went to sleep



No feature flag, no old path: no quick rollback.

The fix: V3, a boring release

1. Deploy V3 behind a feature flag (off), V1 keeps running
2. Flag on, one index type at a time
3. Watch the dashboard: queue depth, throughput, failed rows, staleness
4. Something wrong? Flag off, V1 takes over

I expected fireworks, but it was boring!

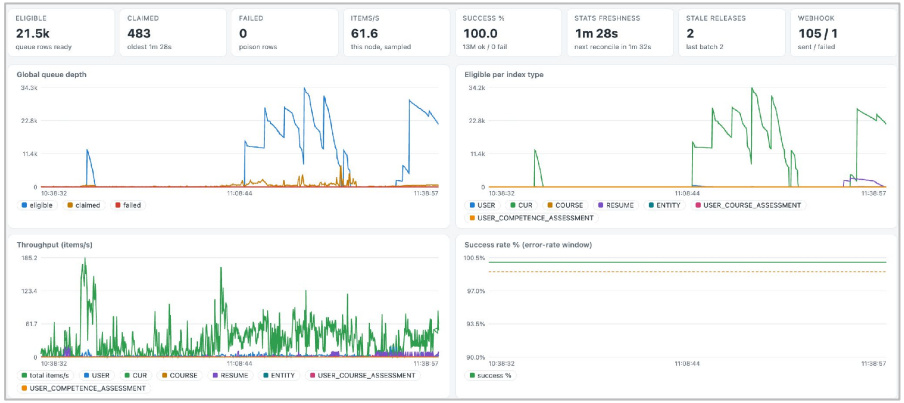


Keep the old path until the new one proves itself.

What production looks like



Draining a backlog of ~800k items



Steady state: 100% success, 0 failed rows

Observability: watch every deployment and release.

Lessons learned

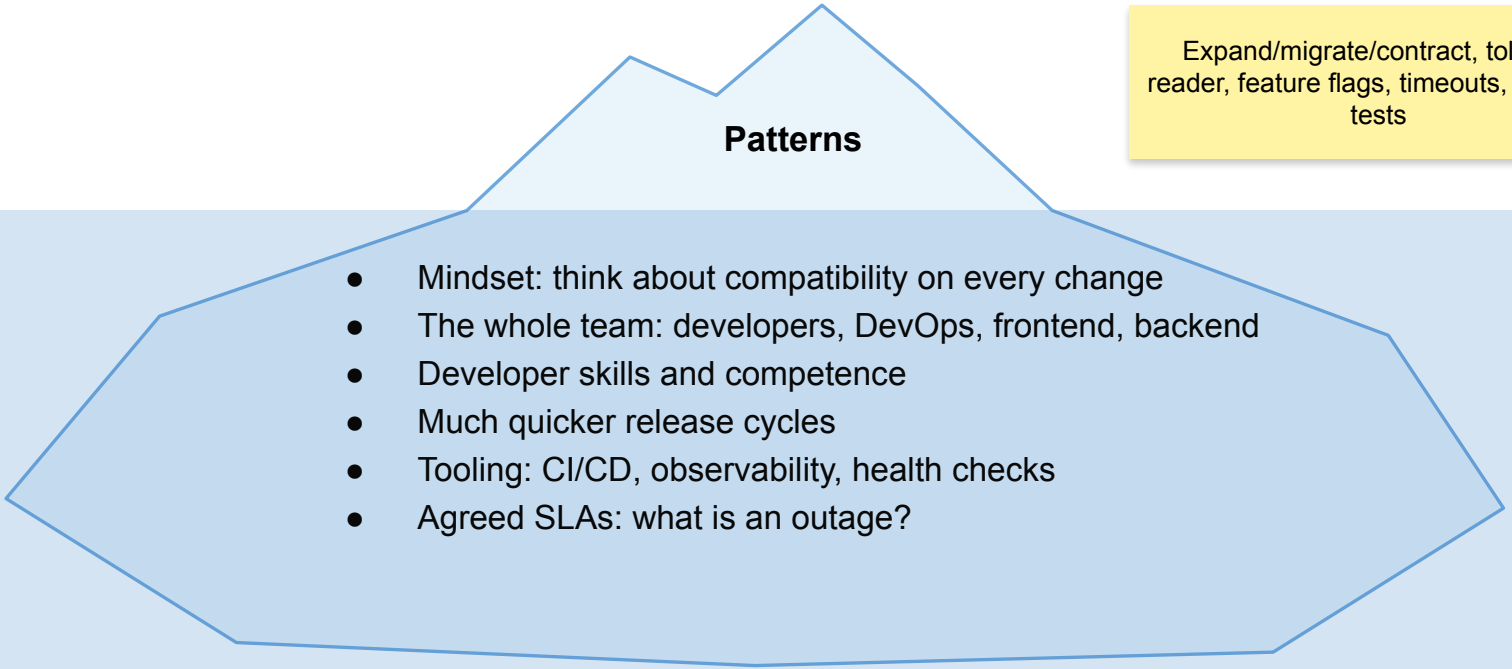
Bulletproof releases: a checklist

- Agree what an outage is, decide if you need zero downtime
- Add first, remove last: expand, migrate, contract
- Backend first, frontend behind a flag
- Every migration has a lock timeout, scary ones a maintenance window
- Risky changes behind a feature flag, keep the old path
- Catch breaking changes in CI (contract tests)



Make every deployment boring.

The tip of the iceberg

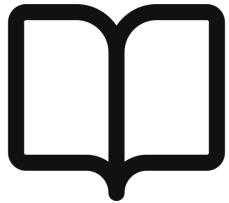


Expand/migrate/contract, tolerant reader, feature flags, timeouts, contract tests

Patterns alone won't do it. It takes the whole team. No more big bang.

Want to go below the surface?

- **Read:** Parallel Change and Tolerant Reader on martinfowler.com
- **Get the slides:** QR code at the end of the talk
- **Take a course:** Implementing Modern Architecture, Part 2 (trainitek.com)
- **Talk to me** after the session, or on LinkedIn



trainitek



marekdominiak

Presentation slides

Please rate my talk

Thank You!

Presentation Slides



marekdominiak

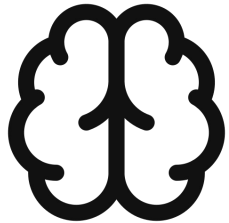


Q/A

Appendix

The proper mindset

- Constantly think about backward and forward compatibility
- Think not only about the code, but about other teams, services and frontends
- Think about the business impact, like availability
- The whole team takes care of API changes



No more big bang.

Maintenance Window

Maintenance Window: Scheduled downtime slot for upgrades, schema migrations, backups, and deep system checks.

Why?

Simplifies Risky Tasks: No need for dual environments or feature flags for complex operations

Clear Communication: Sets user expectations around planned interruptions.

Enables Deep Maintenance: Batch jobs, integrity checks, or cache invalidation without hurry, removal of unnecessary big columns, and tables.

Best practices

Use Maintenance windows for **high-risk or large-scale** work that's hard to split into non-disruptive steps.

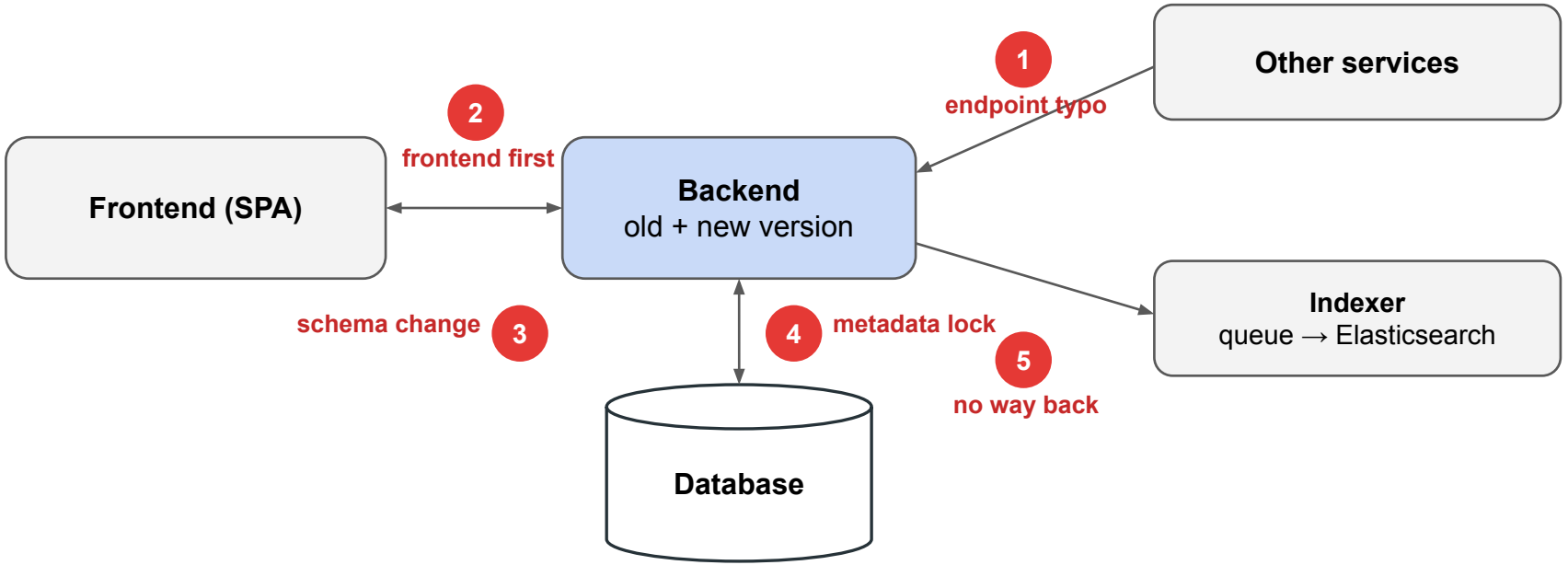
Automation: Pre-checks and **post-validation** scripts to **minimize** window length.

Off-Peak Scheduling: Nights or weekends with the lowest/no usage.

Stakeholder Alerts: Announce well in advance and give progress updates.

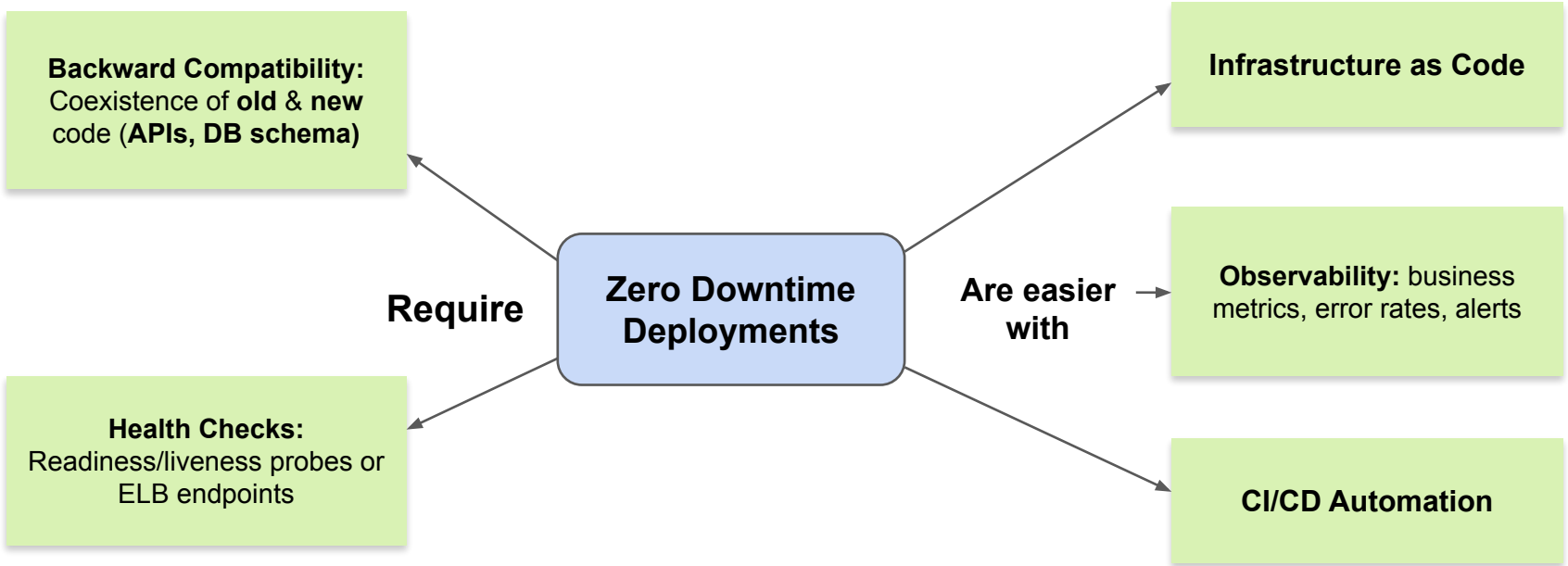
Scary migrations/changes: plan a maintenance window.

Where old and new versions meet



Every arrow is a contract.

Common requirements for all 3 strategies



Tools help with most of it. Backward compatibility is on us.

Sync APIs - Breaking changes

- Deleting or changing the URL of an endpoint (e.g., /customrs → /customers).
- Renaming, removing, or adding required fields.

Old clients aren't aware of any of those changes

Old API Request

```
{
  "userId": 123,
  "email": "user@example.com"
}
```

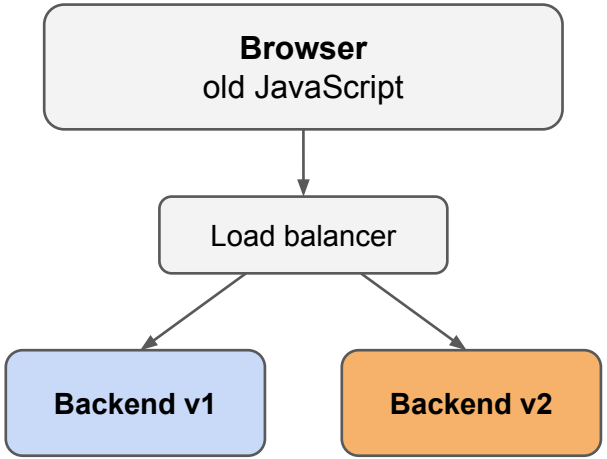
New API Request




```
{
  // Renamed field (userId → id)
  "id": 123,
  "email": "user@example.com"
}
```

Frontend and backend: two versions at once

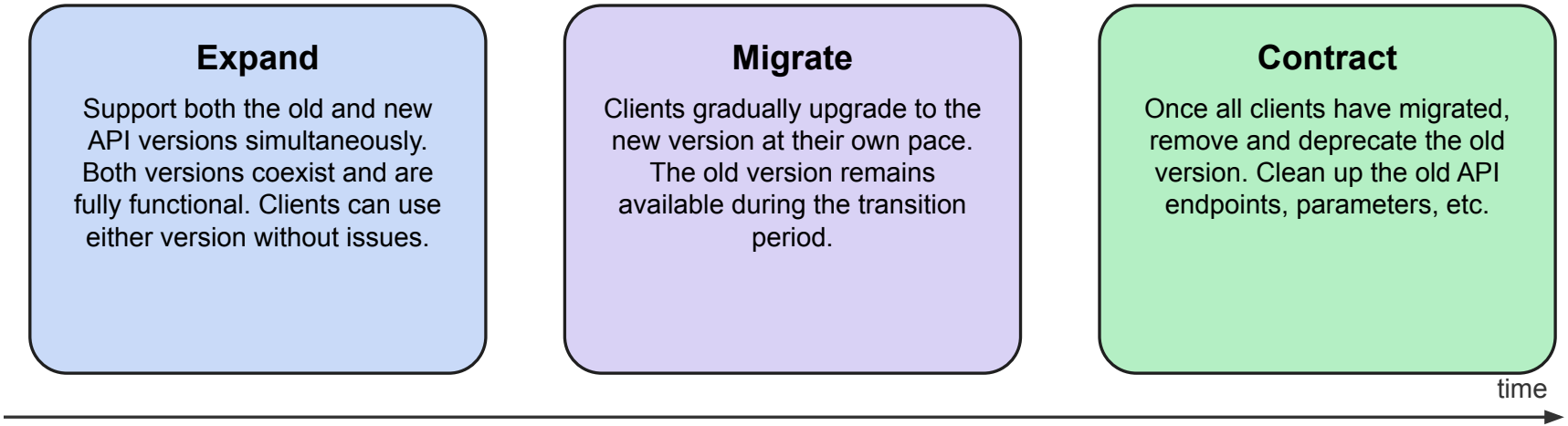
- The SPA in the browser keeps running the old JavaScript until reload
- During a rolling update, some requests hit v1, some hit v2
- Old frontend → new backend, new frontend → old backend
- Generated TypeScript types from OpenAPI can change unexpectedly



Old frontend + new backend must work. And the other way round.

API Evolution: Expand, Migrate, and Contract

- To safely evolve APIs without **breaking** clients, use the **Expand, Migrate, and Contract** (also known as **Parallel Change).**



Expand, Migrate and Contract - requirements

